Python Certificate Projects

Scientific Computing with Python — freeCodeCamp

Project 1: Arithmetic Formatter

A function that arranges arithmetic problems vertically. Handles errors for too many problems, invalid operators, non-digit numbers, and numbers exceeding 4 digits. Optionally displays answers.

Code:

```
def arithmetic_arranger(problems, show_answers=False):
    # spec only allows up to 5 problems at once
    if len(problems) > 5:
        return 'Error: Too many problems.'

    top_row = []
    bottom_row = []
    dash_row = []
    answer_row = []

    for problem in problems:
        parts = problem.split()
        num1 = parts[0]
        operator = parts[1]
        num2 = parts[2]

        if operator not in ['+', '-']:
            return "Error: Operator must be '+' or '-'."

        if not num1.isdigit() or not num2.isdigit():
            return 'Error: Numbers must only contain digits.'

        if len(num1) > 4 or len(num2) > 4:
            return 'Error: Numbers cannot be more than four digits.'

        # +2 leaves room for the operator and a space before the longer number
        width = max(len(num1), len(num2)) + 2
        top_row.append(num1.rjust(width))
        bottom_row.append(operator + num2.rjust(width - 1))
        dash_row.append('-' * width)

        if show_answers:
            if operator == '+':
                answer = str(int(num1) + int(num2))
            else:
                answer = str(int(num1) - int(num2))
            answer_row.append(answer.rjust(width))

    # join each row with two spaces between problems, then stack them
    arranged = ' '.join(top_row) + '\n'
    arranged += ' '.join(bottom_row) + '\n'
    arranged += ' '.join(dash_row)

    if show_answers:
        arranged += '\n' + ' '.join(answer_row)

    return arranged
```

Project 2: Time Calculator

A function that adds a duration to a start time in 12-hour format. Handles AM/PM conversion, day overflow, and optional day-of-week calculation.

Code:

```
 def add_time(start, duration, day=None):
     days_of_week = ['Monday', 'Tuesday', 'Wednesday',
                     'Thursday', 'Friday', 'Saturday', 'Sunday']

     time_part, period = start.split()
     start_hour, start_min = map(int, time_part.split(':'))
     dur_hour, dur_min = map(int, duration.split(':'))

     # convert start time to 24h so the math below is simpler
     if period == 'PM' and start_hour != 12:
         start_hour += 12
     if period == 'AM' and start_hour == 12:
         start_hour = 0

     total_minutes = start_min + dur_min
     extra_hours = total_minutes // 60
     final_min = total_minutes % 60

     total_hours = start_hour + dur_hour + extra_hours
     days_passed = total_hours // 24
     final_hour = total_hours % 24

     # now go back to 12h format for the output
     if final_hour == 0:
         display_hour = 12
         new_period = 'AM'
     elif final_hour < 12:
         display_hour = final_hour
         new_period = 'AM'
     elif final_hour == 12:
         display_hour = 12
         new_period = 'PM'
     else:
         display_hour = final_hour - 12
         new_period = 'PM'

     result = f'{display_hour}:{final_min:02d} {new_period}'

     if day:
         day_index = days_of_week.index(day.capitalize())
         new_day_index = (day_index + days_passed) % 7
         result += f', {days_of_week[new_day_index]}'

     # only mention day rollover if it actually happened
     if days_passed == 1:
         result += ' (next day)'
     elif days_passed > 1:
         result += f' ({days_passed} days later)'

     return result
```

Project 3: Budget App

A Category class for tracking budget with deposit, withdraw, transfer, and get_balance methods. Includes a create_spend_chart function that generates a bar chart of spending by category.

Code:

```
class Category:
    def __init__(self, name):
        self.name = name
        self.ledger = []  # list of dicts: {'amount': ..., 'description': ...}

    def deposit(self, amount, description=''):
        self.ledger.append({'amount': amount, 'description': description})

    def withdraw(self, amount, description=''):
        # only withdraw if there's enough balance, otherwise fail quietly
        if self.check_funds(amount):
            self.ledger.append({'amount': -amount, 'description': description})
            return True
        return False

    def get_balance(self):
        return sum(item['amount'] for item in self.ledger)

    def transfer(self, amount, category):
        # a transfer is just a withdrawal here + a deposit on the other category
        if self.check_funds(amount):
            self.withdraw(amount, f'Transfer to {category.name}')
            category.deposit(amount, f'Transfer from {self.name}')
            return True
        return False

    def check_funds(self, amount):
        return amount <= self.get_balance()

    def __str__(self):
        # matches the required print format: centered title, then ledger lines, then total
        title = self.name.center(30, '*') + '\n'
        items = ''
        for item in self.ledger:
            desc = item['description'][:23]
            amount = f"{item['amount']:.2f}"
            items += f"{desc:<23}{amount:>7}\n"
        total = f"Total: {self.get_balance():.2f}"
        return title + items + total


def create_spend_chart(categories):
    # only withdrawals count as "spent", deposits/transfers in don't
    spent = []
    for cat in categories:
        total = sum(-item['amount'] for item in cat.ledger
                    if item['amount'] < 0)
        spent.append(total)

    total_spent = sum(spent)
    # round each percentage down to the nearest 10 since the chart is drawn in 10% steps
    percentages = [int((s / total_spent) * 100) // 10 * 10 for s in spent]

    chart = 'Percentage spent by category\n'
    for level in range(100, -1, -10):
        line = f'{level:>3}| '
        for pct in percentages:
            line += 'o ' if pct >= level else '  '
        chart += line + '\n'

    chart += '    ' + '-' * (len(categories) * 3 + 1) + '\n'

    # category names are printed vertically, letter by letter, under the bars
    max_len = max(len(cat.name) for cat in categories)
```

```
    for i in range(max_len):
        line = '     '
        for cat in categories:
            if i < len(cat.name):
                line += cat.name[i] + '  '
            else:
                line += '   '
        chart += line + '\n'

    return chart.rstrip('\n')
```

Project 4: Polygon Area Calculator

Rectangle and Square classes using OOP inheritance. Rectangle has methods for area, perimeter, diagonal, picture, and amount_inside. Square inherits from Rectangle and overrides set_width/set_height.

Code:

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def set_width(self, width):
        self.width = width

    def set_height(self, height):
        self.height = height

    def get_area(self):
        return self.width * self.height

    def get_perimeter(self):
        return 2 * self.width + 2 * self.height

    def get_diagonal(self):
        return (self.width ** 2 + self.height ** 2) ** .5

    def get_picture(self):
        # printing anything bigger than 50 would just spam the console
        if self.width > 50 or self.height > 50:
            return 'Too big for picture.'
        return ('*' * self.width + '\n') * self.height

    def get_amount_inside(self, shape):
        # how many times the given shape fits inside this one, grid-style
        return (self.width // shape.width) * (self.height // shape.height)

    def __str__(self):
        return f'Rectangle(width={self.width}, height={self.height})'


class Square(Rectangle):
    def __init__(self, side):
        super().__init__(side, side)

    def set_side(self, side):
        self.width = side
        self.height = side

    # width/height setters are overridden so setting either one keeps it a square
    def set_width(self, width):
        self.width = width
```

```
        self.height = width

    def set_height(self, height):
        self.width = height
        self.height = height

    def __str__(self):
        return f'Square(side={self.width})'
```

Project 5: Probability Calculator

A Hat class that simulates drawing balls without replacement. An experiment function runs multiple trials to estimate the probability of drawing a specific combination of balls.

Code:

```
import copy
import random


class Hat:
    def __init__(self, **kwargs):
        # e.g. Hat(red=3, blue=2) -> contents = ['red','red','red','blue','blue']
        self.contents = []
        for color, count in kwargs.items():
            self.contents.extend([color] * count)

    def draw(self, num_balls):
        # if asking for more balls than exist, just empty the hat
        if num_balls >= len(self.contents):
            drawn = self.contents[:]
            self.contents = []
            return drawn

        drawn = random.sample(self.contents, num_balls)
        for ball in drawn:
            self.contents.remove(ball)
        return drawn


def experiment(hat, expected_balls, num_balls_drawn, num_experiments):
    success = 0

    for _ in range(num_experiments):
        # deep copy so each trial starts with a fresh hat
        hat_copy = copy.deepcopy(hat)
        drawn = hat_copy.draw(num_balls_drawn)

        drawn_counts = {}
        for ball in drawn:
            drawn_counts[ball] = drawn_counts.get(ball, 0) + 1

        # a trial counts as a match only if it hits every expected color/count
        match = all(
            drawn_counts.get(color, 0) >= count
            for color, count in expected_balls.items()
        )
        if match:
            success += 1

    return success / num_experiments
```